Task 1 (Part 1): Student Profile Card

```
import Foundation

let studentName = "Asha"
let rollNumber: Int = 2301
var semester = 5
var cgpa: Double = 8.2
let isHosteller = true

func profileLine(includeCGPA: Bool = false) -> String {
    var line = "Name: \(studentName), Roll: \(rollNumber), Semester: \(semester)"
    if includeCGPA {
        line += ", CGPA: \(String(format: "%.1f", cgpa))"
    }
    return line
}

print(profileLine())

semester += 1
cgpa = 8.5
print(profileLine(includeCGPA: true))
print("Hosteller: \(isHosteller ? "Yes" : "No")")

let values: [(name: String, value: Any)] = [
    ("studentName", studentName),
    ("rollNumber", rollNumber),
    ("semester", semester),
    ("cgpa", cgpa),
    ("isHosteller", isHosteller)
]

for item in values {
    print("\(item.name) is \(type(of: item.value))")
}
```

Output

```
Name: Asha, Roll: 2301, Semester: 5
Name: Asha, Roll: 2301, Semester: 6, CGPA: 8.5
Hosteller: Yes
studentName is String
rollNumber is Int
semester is Int
cgpa is Double
isHosteller is Bool
```

Task 2 (Part 1): Temperature Converter

```
import Foundation

struct Temperature {
   static let fahrenheitFactor = 9.0 / 5.0
   static let fahrenheitOffset = 32.0
   static let kelvinOffset = 273.15

   var celsius: Double

   var fahrenheit: Double {
      celsius * Temperature.fahrenheitFactor + Temperature.fahrenheitOffset
   }

   var kelvin: Double {
      celsius + Temperature.kelvinOffset
   }

   func report() -> String {
      let c = String(format: "%.1f", celsius)
      let f = String(format: "%.1f", fahrenheit)
      let k = String(format: "%.1f", kelvin)
      return "\(c) C = \(f) F = \(k) K"
   }
}

var temperature = Temperature(celsius: 25)
print(temperature.report())

temperature.celsius = 100
print(temperature.report())

for value in [0.0, 37.0, -40.0] {
   print(Temperature(celsius: value).report())
}
```

Output

```
25.0 C = 77.0 F = 298.1 K
100.0 C = 212.0 F = 373.1 K
0.0 C = 32.0 F = 273.1 K
37.0 C = 98.6 F = 310.1 K
-40.0 C = -40.0 F = 233.1 K
```

Task 3 (Part 1): Fixed Compile Errors

```
import Foundation

struct Order {
    var title: String
    var price: Int
    var quantity: Int
    let taxRate: Double

    var subtotal: Double {
        Double(price * quantity)
    }

    var tax: Double {
        subtotal * taxRate
    }

    var total: Double {
        subtotal + tax
    }
}

var order = Order(title: "Order", price: 250, quantity: 3, taxRate: 0.18)
order.price = 300
order.title = "Order 42"

print(order.title)
print("Subtotal: \(String(format: "%.2f", order.subtotal))")
print("Tax: \(String(format: "%.2f", order.tax))")
print("Total including tax: \(String(format: "%.2f", order.total))")
```

Output

```
Order 42
Subtotal: 900.00
Tax: 162.00
Total including tax: 1062.00
```

Task 1 (Part 2): Library Item Model

```
enum BookStatus {
   case available
   case borrowed(by: String, dueIn: Int)
   case reserved(for: String)

   var label: String {
      switch self {
      case .available:
         return "Available"
      case .borrowed(let by, let dueIn):
         return "Borrowed by \(by), due in \(dueIn) days"
      case .reserved(let name):
         return "Reserved for \(name)"
      }
   }
}

struct Book {
   let title: String
   var status: BookStatus = .available
}

extension Book: CustomStringConvertible {
   var description: String {
      "\(title): \(status.label)"
   }
}

class Member {
   var name: String
   private(set) var borrowedBooks: [Book] = []

   init(name: String) {
      self.name = name
   }

   @discardableResult
   func borrow(_ book: Book, days: Int) -> Book {
      var borrowed = book
      borrowed.status = .borrowed(by: name, dueIn: days)
      borrowedBooks.append(borrowed)
      return borrowed
   }
}

let original = Book(title: "Swift Basics")
```

```
var copy = original
copy.status = .reserved(for: "Priya")

print(original)
print(copy)

let member1 = Member(name: "Asha")
let member2 = member1
member2.name = "Meena"

print(member1.name)
print(member2.name)

member1.borrow(original, days: 7)
print(member2.borrowedBooks.map { $0.description })
```

Output

```
Swift Basics: Available
Swift Basics: Reserved for Priya
Meena
Meena
["Swift Basics: Borrowed by Meena, due in 7 days"]
```

Task 2 (Part 2): Shapes and Areas

```
import Foundation

enum Shape {
   case circle(radius: Double)
   case rectangle(width: Double, height: Double)
   case triangle(base: Double, height: Double)

   var name: String {
      switch self {
      case .circle: return "Circle"
      case .rectangle: return "Rectangle"
      case .triangle: return "Triangle"
      }
   }

   var area: Double {
      switch self {
      case .circle(let radius):
         return Double.pi * radius * radius
      case .rectangle(let width, let height):
```

```
            return width * height
        case .triangle(let base, let height):
            return 0.5 * base * height
        }
    }
}

struct Rectangle {
    var width: Double
    var height: Double

    var area: Double {
        width * height
    }

    var perimeter: Double {
        2 * (width + height)
    }

    mutating func scale(by factor: Double) {
        width *= factor
        height *= factor
    }
}

let shapes: [Shape] = [
    .circle(radius: 5),
    .rectangle(width: 4, height: 5),
    .triangle(base: 6, height: 3)
]

for shape in shapes {
    print("\(shape.name) area: \(String(format: "%.2f", shape.area))")
}

let original = Rectangle(width: 4, height: 5)
var copy = original
copy.scale(by: 2)

print("Original: \(original.width) x \(original.height), perimeter \(original.perimeter)")
print("Copy: \(copy.width) x \(copy.height), perimeter \(copy.perimeter)")
```

Output

```
Circle area: 78.54
Rectangle area: 20.00
Triangle area: 9.00
Original: 4.0 x 5.0, perimeter 18.0
Copy: 8.0 x 10.0, perimeter 36.0
```

Task 3 (Part 2): Fixed Counter and Box

```
protocol Incrementable {
   var value: Int { get }
   mutating func increment()
}

struct Counter: Incrementable {
   private(set) var value = 0

   mutating func increment() {
      value += 1
   }
}

final class Box: Incrementable {
   private(set) var value = 0

   func increment() {
      value += 1
   }
}

var c = Counter()
c.increment()

var c2 = c
c2.increment()
print("Counter values: \(c.value) and \(c2.value)")

let b = Box()
b.increment()

let sameBox = b
sameBox.increment()
```

```
let otherBox = Box()

print("Box value: \(b.value)")
print("b === sameBox: \(b === sameBox)")
print("b === otherBox: \(b === otherBox)")
```

Output

```
Counter values: 1 and 2
Box value: 2
b === sameBox: true
b === otherBox: false
```